medical.ge — პროექტის მიმოხილვა

სად რა არის და რას ემსახურება. ონლაინ ჯავშნის საიტი პაციენტებისთვის, ექიმებისთვის, კლინიკებისა და ადმინისტრატორისთვის.

1. ტექნოლოგიები

`Next.js 16 + React 19`	საიტი და სერვერი ერთად: გვერდებიც და API-ც აქ არის
`TypeScript`	JavaScript ტიპებით, ნაკლები შეცდომისთვის
`PostgreSQL + Prisma`	მონაცემთა ბაზა და მასთან მუშაობის ბიბლიოთეკა
`better-auth`	რეგისტრაცია, შესვლა, გასვლა, სესიები
`Zod / Formik`	მონაცემების შემოწმება / ფორმების მართვა
`SCSS Modules`	სტილები, თითო კომპონენტს თავისი ფაილი
`Vitest`	ავტომატური ტესტები

2. მომხმარებლის როლები

პაციენტი (`PATIENT`)	ეძებს ექიმს, ჯავშნის ვიზიტს, ხედავს ისტორიას და დიაგნოზებს
ექიმი (`DOCTOR`)	უერთდება კლინიკას, მართავს განრიგს და პროფილს, წერს დიაგნოზს
კლინიკა (`CLINIC`)	მართავს ექიმებს, კაბინეტებს და ვიზიტებს
ადმინი (`role = admin`)	ამტკიცებს, არედაქტირებს და შლის კლინიკებს

3. მთავარი საქაღალდეები

`prisma/schema.prisma`	ბაზის სტრუქტურა: User, Patient, Doctor, Clinic, Appointment
`prisma/seed.ts`	ბაზის სატესტო მონაცემებით შევსება
`src/app/`	საიტის გვერდები (საქაღალდე = მისამართი)
`src/app/api/`	სერვერის API — ბრაუზერი აქ აგზავნის მოთხოვნებს
`src/components/`	რამდენიმე გვერდზე გამოყენებული ნაწილები
`src/generated/`	Prisma-ს ავტომატურად შექმნილი კოდი — **ხელით არ იცვლება**
`src/lib/`	საერთო ლოგიკა: ბაზა, ავტორიზაცია, დრო, დამხმარე ფუნქციები
`src/styles/`	საერთო ფერები და სტილები
`.env`	ბაზის მისამართი/პაროლი და საიდუმლო გასაღები

> ფრჩხილებში ჩასმული საქაღალდეები, მაგ. `(patient)`, მისამართში არ ჩანს — მხოლოდ გვერდების დასალაგებლადაა. `[doctorId]` ნიშნავს ცვლად ნაწილს, მაგ. `/doctors/123`. `_components` — მხოლოდ ამ გვერდის კომპონენტები.

4. გვერდები (src/app)

`(account)/dashboard`	პირადი კაბინეტი — თითო როლს თავისი მიმოხილვა
`(account)/profile`	პაციენტის პროფილი
`(admin)/admin/clinics, /patients`	ადმინის პანელი
`(auth)/login, /register`	შესვლა და რეგისტრაცია (სამივე როლისთვის)
`(docs)/docs`	API-ის დოკუმენტაცია (Swagger, ქართულად)
`(patient)/booking`	ვიზიტის დაჯავშნა: ფილტრები (სპეციალობა, კლინიკა, ენა), ექიმი, კალენდარი, დრო
`(patient)/my-appointments`	პაციენტის ვიზიტები და ისტორია
`(public)/clinics, /clinics/[id]`	კლინიკების სია და კლინიკის გვერდი
`(public)/doctors, /doctors/[id]`	ექიმების სია და ექიმის საჯარო პროფილი
`(staff)/my-clinic`	კლინიკის მართვა: ექიმები, სტატისტიკა, წაშლა
`(staff)/my-schedule`	ექიმის განრიგი, საათები, პროფილი
`api/`	სერვერის API (იხ. ქვემოთ, პუნქტი 5)
`layout.tsx`	ყველა გვერდის საერთო ჩარჩო: ფონტები და ზედა ზოლი
`page.tsx → /`	მთავარი გვერდი (შიგთავსი: components/home)

5. API (src/app/api)

`admin/clinics/[id], /approve`	ადმინი არედაქტირებს / ამტკიცებს კლინიკას
`appointments (POST)`	ვიზიტის დაჯავშნა (ორმაგი ჯავშნისგან დაცული)
`appointments/[id] (PATCH)`	ვიზიტის გაუქმება ან დიაგნოზის ჩაწერა
`auth/[...all]`	შესვლა / რეგისტრაცია / გასვლა (better-auth)
`clinic-registration`	კლინიკის მონაცემები და ექიმები
`clinics, clinics/[id]`	კლინიკების სია; კლინიკის წაშლა
`doctor-registration`	ექიმის მოთხოვნა კლინიკასთან
`doctors/[id], /profile`	ექიმის საათები/კაბინეტი და პროფილი
`doctors/[id]/approve`	კლინიკა ადასტურებს ან უარყოფს ექიმს

`doctors/[id]/slots`	ექიმის თავისუფალი დროები დღისთვის
`openapi`	API-ის აღწერა Swagger-ისთვის
`patient-registration`	პაციენტის ტელეფონი და სისხლის ჯგუფი

6. საერთო ლოგიკა (src/lib)

`api.ts`	API-ის დამხმარეები: `jsonError`, `readJson`, `isUuid`
`appointments.ts`	ექიმის თავისუფალი დროების გამოთვლა
`auth-client.ts / auth.ts`	ავტორიზაციის ბრაუზერის / სერვერული ნაწილი
`blood-types.ts`	სისხლის ჯგუფების სია
`prisma.ts`	ბაზასთან კავშირი
`request.ts`	`sendRequest()` — ბრაუზერიდან API-ზე გაგზავნა
`session.ts`	`getSession()` — ვინ არის შესული
`slots.ts`	დრო და თარიღი: 30-წუთიანი სლოტები, თბილისის დრო, ფორმატირება
`*.test.ts`	ტესტები (გაშვება: `npm test`)

7. კომპონენტები (src/components)

`appointments/`	ვიზიტის ღილაკები: გაუქმება, დიაგნოზი
`clinics/`	კლინიკის წაშლის ღილაკი (დადასტურებით)
`doctors/`	ექიმის პროფილისა და სამუშაო საათების ფორმები
`home/`	მთავარი გვერდის ნაწილები: Hero, BookingSteps, FeaturedDoctors, JoinBanner
`layout/`	Header (ზედა ზოლი), Sidebar (მენიუ), NavLink, LogoutButton
`ui/`	Calendar, Card, Icon, DoctorAvatar (ფოტო ან ინიციალი)

8. როგორ მუშაობს ჯავშანი

1. პაციენტი `/booking`-ზე ირჩევს ექიმს და დღეს.
2. ბრაუზერი ითხოვს `/api/doctors/[id]/slots` — ბრუნდება მხოლოდ თავისუფალი დროები.
3. დროის არჩევის შემდეგ იგზავნება `POST /api/appointments`.
4. სერვერი ამოწმებს უფლებას, სამუშაო საათებს და დაკავებულობას (ტრანზაქციაში).
5. ვიზიტი ინახება და ჩანს პაციენტის, ექიმისა და კლინიკის გვერდებზე.

9. გაშვება

- `npm install` — ბიბლიოთეკების დაყენება (ერთხელ)
- `npx prisma migrate dev` — ბაზის ცხრილების შექმნა / განახლება
- `npx prisma db seed` — ბაზის სადემონსტრაციო მონაცემებით შევსება
- `npm run dev` — საიტის გაშვება: `http://localhost:3000`
- `npm test` — ტესტები

10. სადემონსტრაციო ანგარიშები

იქმნება `npx prisma db seed`-ით. ყველას პაროლია `demo1234`.

ადმინისტრატორი	admin@medical.ge
კლინიკა	clinic.tbilisi@medical.ge, clinic.batumi@medical.ge
ექიმი	nino.beridze@medical.ge, levan.japaridze@medical.ge
პაციენტი	ana@medical.ge, davit@medical.ge, elene@medical.ge